



Universidad Jorge Tadeo Lozano
Facultad de Ciencias Naturales e Ingenieria

SafeCampus AI

Informe tecnico breve del proyecto de Inteligencia Artificial

Corte 3: Implementacion del Proyecto

Nombre del proyecto	SafeCampus AI: plataforma de seguridad, convivencia y bienestar universitario con IA
Asignatura	Inteligencia Artificial
Profesor	Jorge Ivan Romero Gelvez
Programa academico	Por completar
Grupo / curso	Por completar
Fecha de entrega	09/05/2026

Integrantes	Rol principal en el proyecto
Por completar	Arquitectura, frontend, backend, IA o documentacion
Por completar	Arquitectura, frontend, backend, IA o documentacion
Por completar	Arquitectura, frontend, backend, IA o documentacion
Por completar	Arquitectura, frontend, backend, IA o documentacion

Nota

Este documento describe la implementacion actual del repositorio <https://github.com/Fircrey/SafeCampus>, analizado en el commit 92494e7. Cuando no existen metricas experimentales en el repositorio, el informe las reporta como pendientes o como criterio propuesto de evaluacion, sin inventar resultados.

Datos rapidos del proyecto

Repositorio de GitHub	https://github.com/Fircrey/SafeCampus
Demo funcional	Ejecucion local con Next.js, Flask, Socket.IO, PostgreSQL y modelo YOLOv8 local.
Tipo de problema	Vision por computador, NLP conversacional, recuperacion aumentada por generacion (RAG), gestion de reportes y analitica operativa.
Metrica principal	Para detector: precision, recall y mAP. Para LLM/RAG: exactitud de escalamiento, groundedness, relevancia de respuesta y tasa de alucinacion.
Resultado clave	MVP funcional con autenticacion, reportes georreferenciados, mapa por zonas, detector en vivo, panel administrativo y chat de bienestar con modo demo/OpenAI.

Nota

El sistema es academico y demostrativo. No reemplaza atencion psicologica, seguridad institucional, autoridades universitarias ni servicios de emergencia. La deteccion de armas requiere validacion humana.

1. Resumen ejecutivo

SafeCampus AI es una plataforma universitaria de seguridad, convivencia y bienestar que integra inteligencia artificial en tres frentes: deteccion visual de armas visibles mediante YOLOv8, orientacion conversacional con un LLM y gestion de reportes de situaciones inseguras. El usuario autenticado puede crear reportes asociados a zonas del campus, consultar rutas institucionales y usar un chat de bienestar. Los administradores pueden monitorear el detector en vivo, revisar alertas, gestionar reportes y consultar estadisticas. La arquitectura combina un frontend Next.js, un backend Flask con Socket.IO, PostgreSQL y servicios externos opcionales como OpenAI y Roboflow. Como evolucion del modulo conversacional se propone un LLM con fine-tuning a partir de 500 pares pregunta-respuesta generados por usuario y LLM, complementado con RAG para recuperar reglamentos, protocolos y documentos institucionales vigentes.

Contribucion principal

- Integra en una sola experiencia flujos de reporte, mapa de zonas, administracion, detector de armas y orientacion conversacional.
- Mantiene separacion clara entre frontend serverless-friendly y backend persistente necesario para camara, Socket.IO y YOLOv8.
- Define una ruta de mejora para el chat: pasar de prompt-only con OpenAI a un modelo afinado y aumentado por recuperacion documental.

2. Problema, datos y criterio de exito

2.1. Contexto y objetivo

Las universidades requieren mecanismos rapidos para identificar riesgos, orientar a estudiantes y canalizar reportes hacia equipos humanos. En un campus, los eventos de seguridad o bienestar pueden depender de ubicacion, urgencia, confidencialidad, evidencia visual y rutas institucionales. SafeCampus AI busca apoyar esa primera capa de deteccion, orientacion y trazabilidad sin sustituir la toma de decision humana.

Usuario objetivo: estudiantes, personal administrativo, seguridad institucional y equipo de bienestar universitario.

Objetivo medible: reducir friccion para reportar incidentes, mejorar trazabilidad por zona y alertar oportunamente a administradores ante detecciones visuales o reportes de alto riesgo.

Alcance de esta version: MVP demostrativo con autenticacion, roles, dashboard, reportes, mapa SVG, detector en vivo, administracion y chat de bienestar. Quedan por fuera una evaluacion cuantitativa formal del detector, auditoria clinica del chat, despliegue productivo completo y migraciones de base de datos.

2.2. Datos utilizados

Fuente	Uso dentro del sistema
Documentacion institucional de Utadeo	Contexto del asistente de bienestar: reglamento estudiantil, protocolos de genero/acoso sexual, buen trato y rutas de contacto.
Zonas del campus en <code>utadeo-zones.json</code>	Mapa SVG, seleccion de zona, conteo de reportes activos y creacion de reportes georreferenciados.
Reportes creados por usuarios	Persistencia operativa en PostgreSQL; alimentan dashboard, panel admin y mapa por zonas.
Stream de camara local	Entrada visual para el detector YOLOv8 en Flask/OpenCV.
Modelo YOLOv8 / Roboflow Universe	Deteccion de armas visibles, como pistola o cuchillo, con umbral configurable.
Dataset propuesto de 500 QA	Fine-tuning futuro del LLM SafeCampus: tono, escalamiento, respuestas frecuentes y criterios de seguridad.

2.3. Criterios de exito

Modulo	Criterio de exito
Reportes	Crear, enriquecer, consultar y administrar reportes con estado, prioridad, zona, foto opcional y trazabilidad.
Detector	Procesar stream de camara, dibujar bounding boxes, emitir eventos alert , new_detection y stats_update .
Chat/LLM	Responder en espanol, orientar sin diagnosticar, escalar a canales humanos ante emergencia y usar contexto institucional recuperado por RAG.
Seguridad	Restringir rutas y endpoints por JWT y rol: user , admin , superadmin .
Reproducibilidad	Ejecutar frontend, backend y base de datos localmente con instrucciones y variables documentadas.

3. Implementacion

3.1. Arquitectura general

El proyecto esta organizado como una aplicacion full-stack con dos runtimes principales. Next.js sirve la interfaz, rutas de aplicacion y endpoints auxiliares para OpenAI/Roboflow. Flask implementa autenticacion, reportes, administracion, detector, video stream y Socket.IO. PostgreSQL almacena usuarios y reportes.

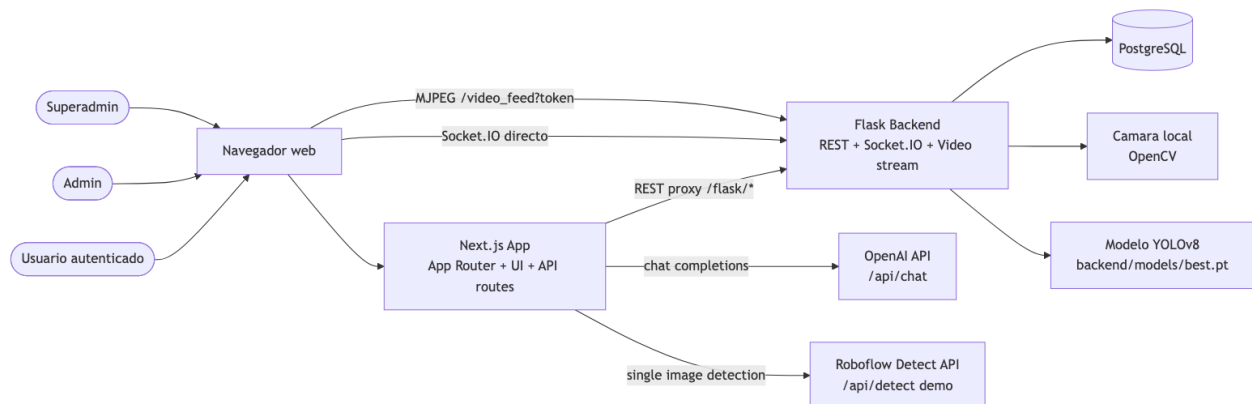


Figura 1: Contexto general de SafeCampus AI.

3.2. Componentes de software

- **Frontend:** Next.js 16, React 19, TypeScript, Tailwind CSS, componentes por dominio y middleware de rutas protegidas.
- **Backend:** Flask, Flask-SQLAlchemy, Flask-SocketIO, Flask-CORS, PyJWT, bcrypt, OpenCV y

Ultralytics.

- **Persistencia:** PostgreSQL 16 mediante Docker Compose; tablas **users** y **reports**.
- **Tiempo real:** Socket.IO para alertas, estadísticas, historial de detección y nuevos reportes por zona.
- **IA:** YOLOv8 local para vision, OpenAI para chat, Roboflow como endpoint demo o inferencia alternativa.

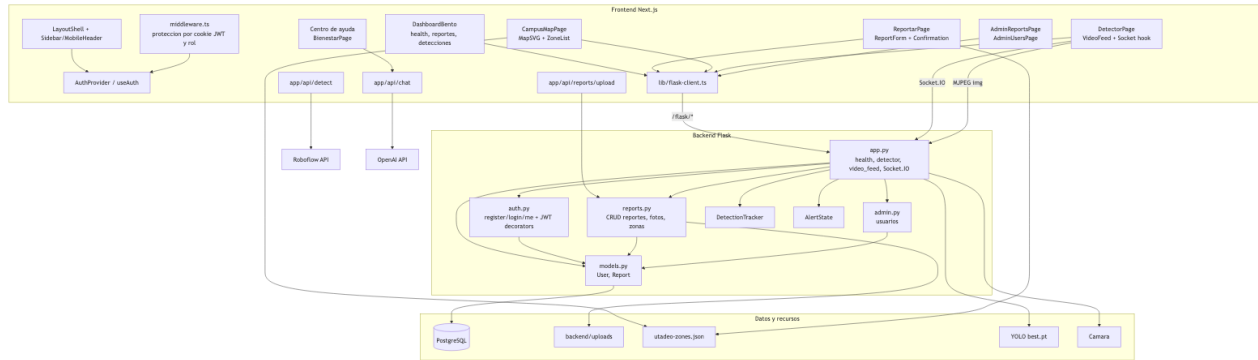


Figura 2: Componentes principales de la implementación.

3.3. Arquitectura LLM propuesta

El modulo conversacional actual usa `/api/chat`, instrucciones institucionales y OpenAI. La mejora propuesta incorpora:

1. **Fine-tuning:** un conjunto de 500 pares pregunta-respuesta generados colaborativamente por el usuario y un LLM, luego curados por el equipo.
2. **RAG:** documentos institucionales se normalizan, dividen en fragmentos, convierten a embeddings y se almacenan en un vector store.
3. **Orquestacion:** cada consulta se convierte a embedding, recupera contexto relevante, aplica guardrails y finalmente consulta el LLM afinado.

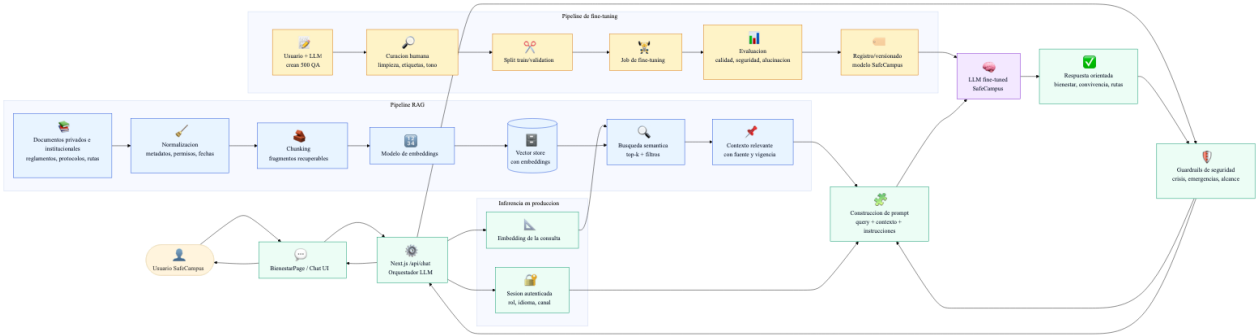


Figura 3: Arquitectura propuesta del LLM con fine-tuning y RAG.

3.4. Baseline vs. modelo final

Aspecto	Baseline	Modelo / sistema final propuesto
Chat	Prompt-only con OpenAI y contexto escrito en <code>lib/protocols.ts</code> .	LLM fine-tuned con 500 QA + RAG sobre documentos institucionales versionados.
Vision	Simulacion o endpoint Roboflow para imagen puntual.	Detector YOLOv8 local en Flask con camara, bounding boxes, eventos Socket.IO e historial.
Reportes	Formulario simple sin mapa.	Reportes georreferenciados, seleccion de zona, foto opcional, enriquecimiento y panel admin.
Administracion	Sin roles avanzados.	Roles <code>user</code> , <code>admin</code> , <code>superadmin</code> ; rutas y endpoints protegidos.
Limitacion	Baja trazabilidad y poca adaptacion al contexto institucional.	Requiere evaluacion cuantitativa, despliegue persistente y hardening de seguridad.

3.5. Decisiones tecnicas clave

Decision tecnica

Separar Next.js y Flask. Next.js se mantiene enfocado en UI y APIs ligeras; Flask concentra video, modelo YOLO, Socket.IO y persistencia. Esta separacion evita intentar ejecutar camara o WebSockets persistentes en entornos serverless.

Decision tecnica

Usar roles y JWT. La aplicacion requiere diferentes niveles de acceso: usuario, administrador y superadministrador. El middleware de Next mejora la experiencia de navegacion, mientras Flask verifica realmente autorizacion en endpoints.

Decision tecnica

Combinar fine-tuning y RAG para el LLM. El fine-tuning enseña estilo, tono y politicas de escalamiento; RAG mantiene actualizados reglamentos, protocolos y contactos sin reentrenar el modelo ante cada cambio documental.

4. Resultados, analisis y demo

4.1. Estado de resultados

El repositorio analizado no incluye una suite de pruebas automatizadas ni resultados cuantitativos de entrenamiento/evaluacion. Por rigor, este informe separa lo implementado de lo que debe medirse antes de afirmar desempeno del modelo.

Elemento	Estado	Evidencia
Autenticacion y roles	Implementado	<code>backend/auth.py</code> , <code>middleware.ts</code> , <code>AuthContext.tsx</code> .
Reportes	Implementado	<code>backend/reports.py</code> , <code>components/reportar/*</code> , <code>components/admin/AdminReportsPage.tsx</code> .
Mapa por zonas	Implementado	<code>components/mapa/*</code> , <code>lib/data/utadeo-zones.json</code> , evento <code>new_zone_report</code> .
Detector en vivo	Implementado con dependencia de <code>best.pt</code>	<code>backend/app.py</code> , <code>components/detector/*</code> .
Chat de bienestar	Implementado en modo OpenAI/demo	<code>app/api/chat/route.ts</code> , <code>components/chat/BienestarPage.tsx</code> .
Fine-tuning LLM	Propuesto	Requiere dataset curado de 500 QA, job de entrenamiento y evaluacion.
RAG	Propuesto	Requiere vector store, embeddings y pipeline documental.
Evaluacion cuantitativa	Pendiente	No hay tests ni notebooks de metricas en el repositorio clonado.

4.2. Metricas recomendadas

Modulo	Metricas	Justificacion
Detector YOLOv8	Precision, recall, F1, mAP@0.5, falsos positivos por hora.	En seguridad importa detectar riesgos sin saturar al equipo con alertas incorrectas.
Chat LLM	Relevancia, groundedness, exactitud de escalamiento, tasa de alucinacion, satisfaccion de usuario.	El asistente debe orientar con contexto institucional y activar canales humanos cuando corresponda.
RAG	Recall@k, MRR, precision de fuentes, latencia de recuperacion.	El sistema debe recuperar documentos correctos y vigentes antes de responder.
Reportes	Tiempo de creacion, tasa de formularios completos, reportes por zona, tiempo de resolucion.	Permite evaluar utilidad operativa, no solo desempeno del modelo.

4.3. Analisis de riesgos

1. **Falsos negativos en vision:** si el detector no identifica un arma visible, el sistema podria no alertar al administrador. Mitigacion: validacion humana, umbrales calibrados y evaluacion con dataset representativo.
2. **Falsos positivos:** alertas repetidas o incorrectas pueden generar fatiga operativa. Mitigacion: cooldown, debounce de frames limpios y revision humana.
3. **Alucinaciones del LLM:** el chat podria inventar rutas o politicas. Mitigacion: RAG con fuentes, citas, guardrails y evaluacion con casos de riesgo.
4. **Datos sensibles:** reportes, fotos y ubicaciones pueden contener informacion personal. Mitigacion: control por rol, tokens, almacenamiento seguro y politicas de retencion.

4.4. Demo funcional

Tecnologia usada	Next.js, React, Flask, Socket.IO, PostgreSQL, YOLOv8, OpenAI y Roboflow opcional.
Entrada del usuario	Credenciales, mensajes de chat, reportes, ubicacion/zona, fotos opcionales y controles del detector.
Salida del sistema	Respuestas de bienestar, reportes persistidos, conteos por zona, alertas en tiempo real, stream con bounding boxes y panel admin.
Casos mostrados	Login/registro, crear reporte, complementar reporte, mapa por zona, detector en vivo, administracion de reportes/usuarios.
Limitaciones visibles	Modo demo si faltan llaves, dependencia del modelo local <code>best.pt</code> , backend persistente requerido y sin reemplazo de canales humanos.

5. Cierre y reproducibilidad

5.1. Instrucciones de ejecucion local

```
npm install
npm run dev:db
npm run dev:flask
npm run dev
```

Variables principales:

```
OPENAI_API_KEY=
OPENAI_MODEL=gpt-4o-mini
ROBOFLOW_API_KEY=
ROBOFLOW_MODEL_ID=weapon-detection-using-yolov8/1
NEXT_PUBLIC_FLASK_URL=http://localhost:5000
DATABASE_URL=postgresql://safecampus:safecampus123@localhost:5433/safecampus
```

5.2. Checklist de reproducibilidad

Criterio	Listo	Evidencia / ruta
El repositorio tiene README con instrucciones claras.	✓	README.md.
Las dependencias frontend estan documentadas.	✓	package.json.
Las dependencias backend estan documentadas.	✓	backend/requirements.txt.
La base de datos local esta definida.	✓	docker-compose.yml.
La demo ejecuta inferencia real del detector si existe <code>best.pt</code> .	✓	backend/app.py.
El modulo LLM fine-tuned + RAG esta implementado.	□	Propuesto en arquitectura; falta pipeline productivo.
Hay pruebas automatizadas y metricas experimentales.	□	No existen en el commit analizado.

5.3. Conclusiones

1. SafeCampus AI logra un MVP integrado que combina IA, reportes, mapa y administracion con una arquitectura comprensible y extensible.
2. La decision de separar Next.js y Flask es adecuada por las necesidades de camara, stream MJPEG, Socket.IO y modelo YOLO local.
3. El chat actual es util para demostracion, pero la arquitectura LLM con fine-tuning y RAG es necesaria para mejorar consistencia, trazabilidad y actualizacion documental.
4. La principal brecha tecnica es la ausencia de evaluacion cuantitativa: se deben agregar pruebas, dataset de validacion, metricas del detector y evaluacion del LLM antes de uso real.

6. Referencias bibliograficas

Referencias

- [1] Fircrey. (2026). *SafeCampus*. GitHub. <https://github.com/Fircrey/SafeCampus>
- [2] Universidad Jorge Tadeo Lozano. *Paleta de colores y elementos graficos de marca*. <https://www.utadeo.edu.co/es/marca/elementos-graficos/paleta-de-colores>
- [3] Universidad Jorge Tadeo Lozano. *Reglamento estudiantil*. <https://www.utadeo.edu.co/es/link/bienestar-universitario/72301/reglamento-estudiantil>
- [4] Universidad Jorge Tadeo Lozano. *Protocolo para la prevencion, deteccion y atencion de violencias por razones de genero y/o violencia sexual*. <https://www.utadeo.edu.co/es/link/bienestar-universitario/72301/protocolo-para-la-prevencion-deteccion-y-atencion-de-violencias-por-razones-de-genero-y-o-violencia-sexual>

[//www.utadeo.edu.co/es/noticia/destacadas/bienestar-universitario/72301/protocolo-para-la-prevencion-deteccion-y-atencion-de-casos-de-violencias-por-razones-de-genero](https://www.utadeo.edu.co/es/noticia/destacadas/bienestar-universitario/72301/protocolo-para-la-prevencion-deteccion-y-atencion-de-casos-de-violencias-por-razones-de-genero)

[5] Roboflow Universe. *Weapon Detection using YOLOv8*. <https://universe.roboflow.com/weapon-detection/weapon-detection-using-yolov8>

[6] Ultralytics. *YOLO Documentation*. <https://docs.ultralytics.com/>

[7] OpenAI. *API Documentation*. <https://platform.openai.com/docs>

A. Diagramas adicionales

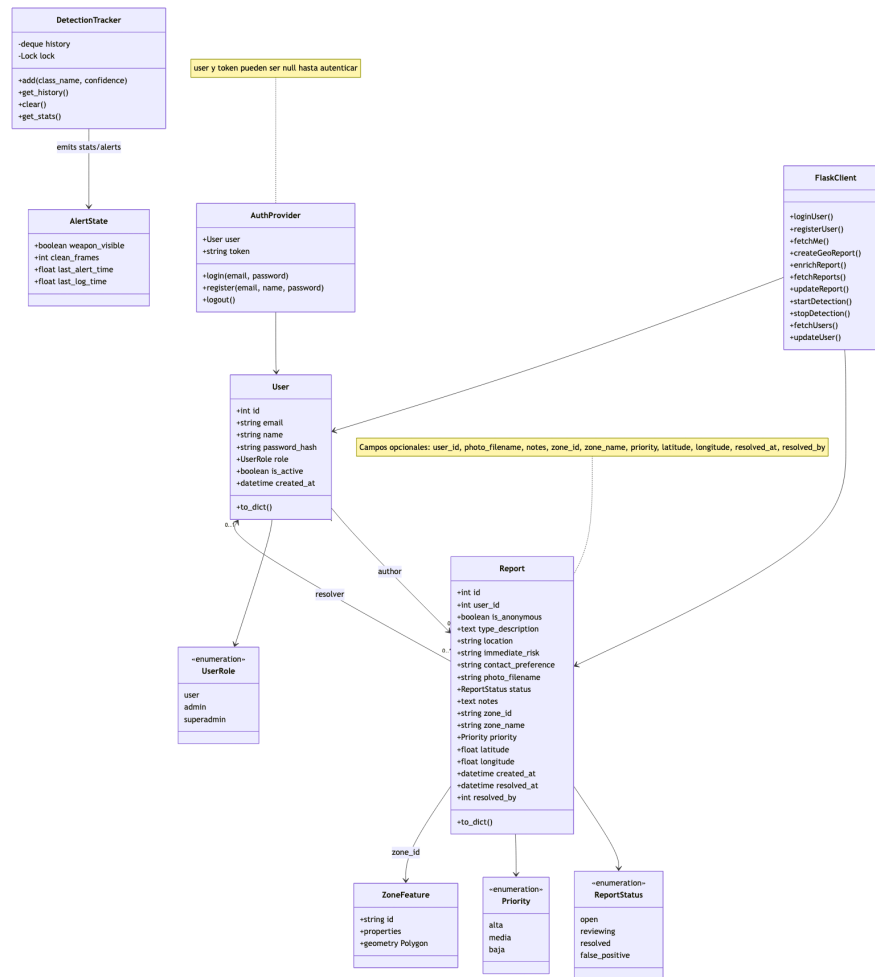


Figura 4: Modelo de clases y dominio.

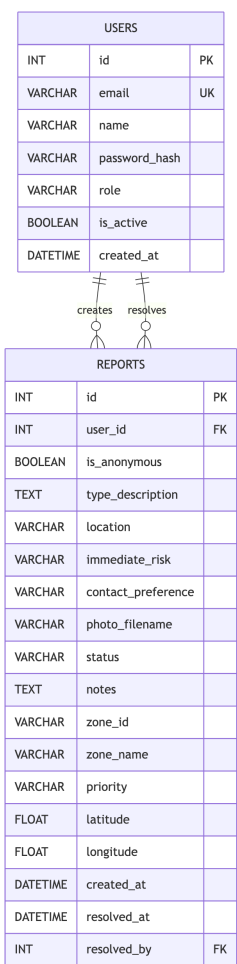


Figura 5: Entidad-relacion de usuarios y reportes.

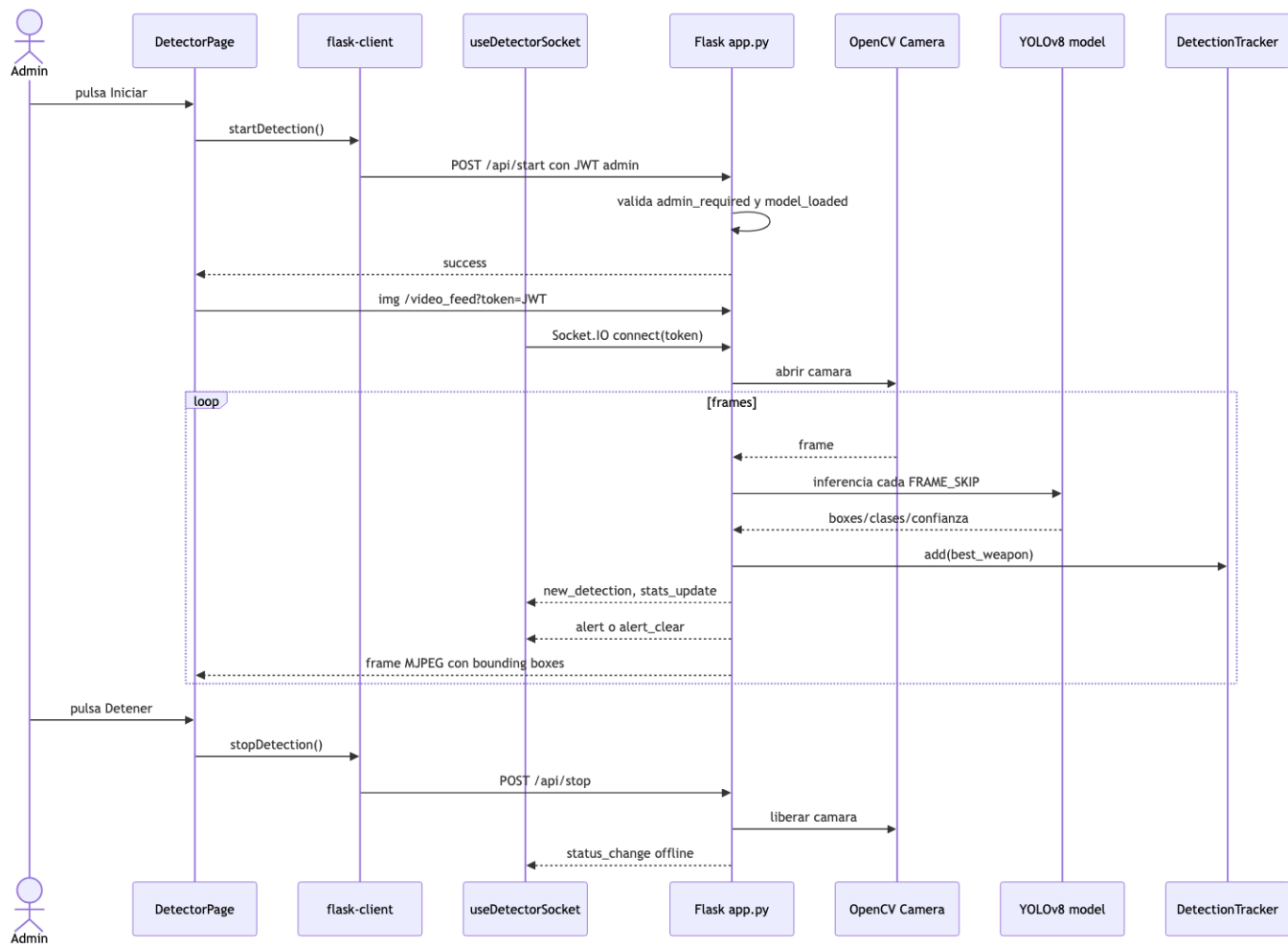


Figura 6: Secuencia del detector en vivo.

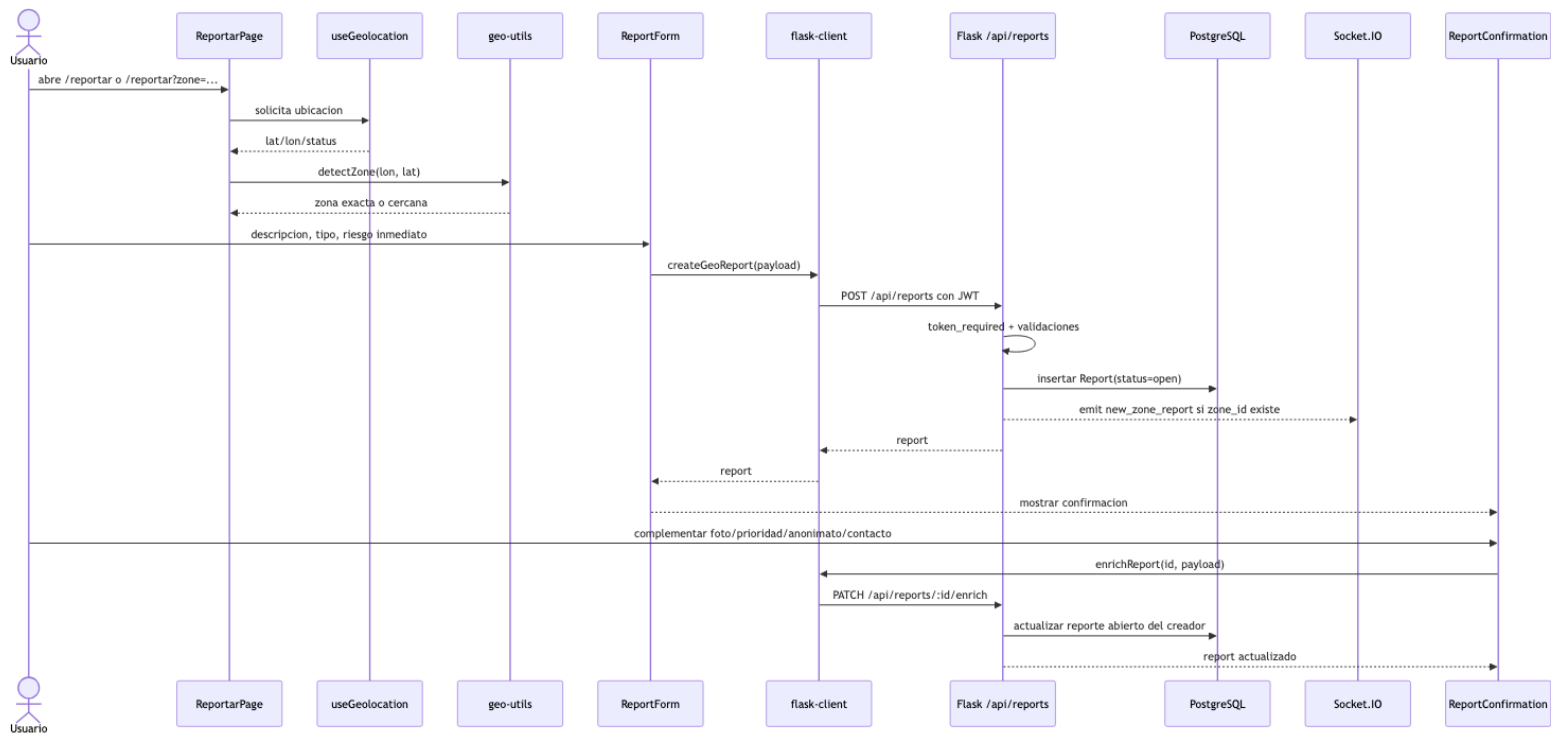


Figura 7: Secuencia de creacion y enriquecimiento de reportes.

B. Enlaces finales

Recurso	Enlace o ruta
Repositorio de GitHub	https://github.com/Fircrey/SafeCampus
Documento de arquitectura	docs/diagramas-arquitectura.md
Diagramas renderizados	docs/rendered/
Backend Flask	backend/
Frontend Next.js	app/, components/, lib/